VeriGate: An Adaptive, Self-Verifying Semantic Cache for Scalable and Cost-Efficient LLM Gateways

A report submitted for the course named Project - II (CS4201)

Submitted By

ABHISHEK YADAV
SEMESTER - VII
230103041

Supervised By

DR. SALAM MICHAEL SINGH



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SENAPATI, MANIPUR

NOVEMBER, 2026

Declaration

In this submission, I have expressed my idea in my own words, and I have adequately cited and referenced any ideas or words that were taken from another source. I also declare that I adhere to all principles of academic honesty and integrity and that I have not misrepresented or falsified any ideas, data, facts, or sources in this submission. If any violation of the above is made, I understand that the institute may take disciplinary action. Such a violation may also engender disciplinary action from the sources which were not properly cited or permission not taken when needed.

ABHISHEK YADAV
230103041

Department of Computer Science and Engineering
Indian Institute of Information Technology Senapati, Manipur

Dr. Salam Michael Singh
Assistant Professor

Email: michael_s@iiitmanipur.ac.in
Contact No: +91 8575443081

To Whom It May Concern

This is to certify that the project report entitled **"VeriGate: An Adaptive, Self-Verifying Semantic Cache for Scalable and Cost-Efficient LLM Gateways",** submitted to the department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur in partial fulfillment for the award of degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of work carried out by **ABHISHEK YADAV** bearing roll number 230103041.

Signature of Supervisor

(Dr. Salam Michael Singh)

Signature of the Examiner 1 .....

Signature of the Examiner 2 .....

Signature of the Examiner 3 .....

Signature of the Examiner 4 .....

Abstract

VeriGate is an LLM gateway with an experimental semantic answer cache. Its conservative default reuses only exact prompts; approximate reuse is opt-in. The gateway also provides tenant separation, rate limiting, provider routing, streaming, and observability. This report documents the implementation and a small candidate-pair smoke experiment. That experiment does not measure whether a specific cached answer remains correct on a new request, and its results do not establish a production error rate or a novel algorithmic contribution. The research direction is to evaluate answer-aware, cost-aware selective verification against fairly tuned baselines on independently reviewed, held-out data.

Acknowledgement

I would like to express my deepest appreciation to all those who provided me the possibility to complete this project. A special gratitude I give to my supervisor, **Dr. Salam Michael Singh**, whose contribution in stimulating suggestions and encouragement helped me to coordinate my project especially in writing this report.

I would also like to express my sincere gratitude to the Head of the Department (CSE), **Dr. Navanth Saharia**, for his constant support and for providing the necessary facilities to successfully execute this project.

Furthermore, I would also like to acknowledge with much appreciation the crucial role of the faculty and staff of the Department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur, who gave the permission to use all required equipment and necessary materials to complete the tasks.

Last but not least, many thanks go to my family and friends whose support, guidance, and encouragement have been invaluable throughout this endeavor.

ABHISHEK YADAV
Roll No: 230103041

Contents

Abstract **3**

Acknowledgement **4**

List of Figures **8**

List of Tables **10**

1 Introduction **11**

1.1 Background 11

1.2 Motivation 11

1.3 Problem Definition 12

1.4 Project Objectives 12

1.5 Scope of the Project 13

1.6 Project Timeline 13

2 Literature Survey **15**

2.1 Scope of the Comparison 15

2.2 Directly Relevant Systems 15

2.3 Research Position 16

3 Requirement Engineering **17**

3.1 Introduction 17

3.2 System Analysis 17

3.2.1 Limitations of the Existing System 17

3.2.2 Feasibility Study 18

3.3 System Requirements 18

3.3.1 Hardware Requirements 18

3.3.2 Software Requirements 18

3.4 Functional Requirements 19

3.5 Non-Functional Requirements 20

4 System Design **21**
4.1 Introduction to System Design . . . 21
4.2 Design Goals and Guiding Principles . . . 21
4.3 Design Patterns Employed . . . 22
4.4 System Architecture . . . 22
4.5 Use Case Analysis . . . 23
4.6 Data Flow Diagrams . . . 24
4.6.1 Level-0 DFD (Context Diagram) . . . 24
4.6.2 Level-1 DFD (Process Decomposition) . . . 25
4.7 Request Lifecycle and the Cache Decision . . . 25
4.7.1 Request Sequence (HIT vs MISS) . . . 27
4.8 Cache Intelligence Design . . . 27
4.8.1 Adaptive Similarity Threshold . . . 27
4.8.2 Two-Tier Self-Verifier . . . 27
4.8.3 Volatility Detection . . . 28
4.9 Cost-Aware Cascade Design . . . 28
4.10 Component and Module Design . . . 28
4.11 Data Model and Tenant Namespacing . . . 29
4.12 API Route Design . . . 30
4.13 Deployment Topology . . . 30
4.14 Failure-Mode and Error-Handling Design . . . 31

5 Implementation **33**
5.1 Introduction . . . 33
5.2 Architectural Decisions . . . 33
5.3 Configuration . . . 34
5.4 The Cache Policy . . . 34
5.4.1 Adaptive Threshold and Calibration . . . 34
5.4.2 Tier-1 Structural Verifier . . . 35
5.4.3 Tier-2 NLI Verifier . . . 35
5.5 Semantic Cache and Vector Index . . . 35
5.6 Provider Abstraction, Cascade, and Circuit Breaker . . . 35
5.7 Production Hardening . . . 36
5.8 Observability and Operator Dashboard . . . 36
5.9 Testing . . . 37

6 Results and Evaluation **38**
6.1 Functional Checks . . . 38

6.2 Candidate-Pair Smoke Fixture 38
6.3 Stage and Backend Timings 39
6.4 Illustrative Cascade Experiment 40
6.5 Validity and Next Experiment 40

7 Conclusion **41**

Bibliography **42**

List of Figures

1.1 VeriGate development timeline across twelve weekly sprints, colour-coded by phase: Foundation (blue), Cache Core (teal), Performance (orange), Providers and Cascade (purple), and Hardening and Evaluation (red). 14

4.1 VeriGate four-layer architecture. Requests descend from the edge; a verified cache hit in Layer 2 (highlighted) short-circuits the provider call in Layer 3. All state in Layer 4 is namespaced per tenant. 23

4.2 UML use-case diagram. The Client Application submits and streams queries; every submission *includes* a semantic lookup and a verify step, and *extends* to a model route only on a miss. The Operator manages and clears the cache; the external LLM Provider fulfils missed calls. 24

4.3 Level-0 (context) DFD. The gateway exchanges queries and answers with the Client, prompts and completions with the external LLM Provider (only on a cache miss), and control/metrics with the Operator. 24

4.4 Level-1 DFD. The main data flow runs left to right along one lane (Client → P1 → P2 → P3 → P4 → Provider); data stores connect by vertical (orthogonal) flows. A verified hit in P3 returns the answer to the client; only a miss reaches P4 and the external provider, whose answer is written back to the cache (D1). 25

4.5 The four-outcome cache decision. A candidate whose similarity clears the adaptive threshold is still subjected to the verifier; only a verified candidate is served as a HIT, while a rejected one is demoted to a MISS ("blocked"). Volatile queries bypass the cache entirely. 26

4.6 Request sequence. A cold query misses the cache, is answered by the provider (~800 ms), and is stored. A subsequently reworded query clears the adaptive threshold, passes verification, and is served from cache (~10 ms, $0) without touching the provider — the core cost/latency win. Solid arrows are calls; dashed arrows are returns; bars are activations. 27

4.7 Cost-aware cascade. A transparent complexity score routes easy queries to the cheap model and escalates only hard ones to the strong model; the unused tier remains as a failover. 28

4.8 Production deployment topology. Stateless gateway replicas ($\times N$) sit behind a TLS-terminating load balancer and share one Redis for all state, so the cache, budgets, and breaker are consistent across replicas; Prometheus/Grafana observe every replica. Each connector uses an orthogonal path on its own side of the gateway. 31

6.1 Threshold sweep on the candidate-pair smoke fixture. Axes are negative-pair acceptance and positive-pair recall, not real workload risk and savings. 39

List of Tables

2.1 Verified scope of directly relevant caching approaches. A checkmark describes a reported method, not an independently reproduced result in this project. 16

4.1 Design patterns used in VeriGate and the design goal each one serves. 22

4.2 Module responsibilities mapped to architectural layers. The contribution is concentrated in `app/cache/`, reached through the single `CachePolicy` object. 29

4.3 Namespaced state in Redis. Every cache and index key is scoped by `tenant_id`, so one tenant can never read another's entries. 30

4.4 Primary API routes. † `/metrics` is public in the demo build and is flagged for lock-down before any multi-tenant public deployment (Chapter 5, security). 30

4.5 Fault-tolerance matrix: how each anticipated fault is detected and handled without surfacing an error to the caller. 32

6.1 Default local pytest run; skipped integration tests require a disposable Redis Stack. 38

6.2 Candidate-pair smoke fixture. Positive recall and negative acceptance use different denominators; this is not workload hit rate or monetary savings. 39

6.3 Recorded stage timings in milliseconds; these are not end-to-end provider latency. . 40

6.4 Recorded lookup p50 latency in milliseconds; the crossover size was not measured. . 40

6.5 Illustrative cascade cost model. Quality retention is hard-label recall, not judged answer quality. 40

Chapter 1

Introduction

1.1 Background

Large Language Models (LLMs) such as the GPT and Gemini families have moved rapidly from research artifacts into production software, powering chat assistants, retrieval-augmented question answering, code generation, and agentic workflows. The transformer architecture that underpins them [17, 4] delivers remarkable general-purpose language capability, but it does so at a recurring price: every request to a hosted LLM incurs a per-token monetary cost and a network-and-inference latency that is frequently measured in seconds rather than milliseconds. At the scale of a real application serving thousands of users, these two quantities — cost and latency — become the dominant operational constraints, and reducing them without degrading answer quality is a central problem in efficient LLM serving [19].

A widely adopted mitigation is the **semantic cache**. A conventional cache keys on an exact string, so it only helps when the identical request repeats. A semantic cache instead embeds the incoming query into a dense vector [14] and, if a previously answered query lies "close enough" in that vector space, returns the stored answer without calling the LLM at all. Because natural-language questions are frequently reworded (*"how do I reset my password"* versus *"what is the process to recover my account password"*), a semantic cache captures far more reuse than an exact cache, converting a slow, paid model call into an instant, free lookup. Open-source systems such as GPTCache [1] have popularised this pattern.

However, the semantic cache carries a failure mode that the exact cache does not. Because it serves an answer whenever similarity crosses a threshold, two questions that are lexically and semantically *close* but factually *distinct* can collide. The canonical example is *"what is the capital of Austria"* versus *"what is the capital of Australia"*: the two sentences differ by two characters, embed very near each other, and yet demand entirely different answers. When a fixed-threshold cache serves the stored "Vienna" answer to the "Australia" query, it commits a **false cache hit** — confidently returning a wrong answer. This is more dangerous than a cache *miss*, because a miss merely forfeits a saving whereas a false hit silently corrupts correctness.

1.2 Motivation

The motivation for this project is the observation that the quality of a semantic cache is governed almost entirely by a single, brittle hyper-parameter: the similarity threshold. Set it too high and the cache rarely fires, forfeiting the cost and latency savings that justify its existence. Set it too low and false hits proliferate. Crucially, *no single fixed threshold is correct for all queries.* The ideal threshold depends on how densely the neighbourhood around a query is populated, on how

factually load-bearing the query's tokens are (a swapped digit or a negation flips the answer), and on whether the query is even cacheable at all (a request for *today's* weather must never be served from yesterday's cache).

Existing systems treat this as a configuration value the operator must guess, and they trust the similarity score alone to decide a hit. There is therefore a clear gap between the *detection* of a nearby cached answer and the *verification* that the answer is actually correct for the new question. Bridging that gap — making the cache adapt its strictness per query and *verify* a candidate hit before serving it — is the intellectual core of this work. A cache that can decline to answer when it is not sure, rather than confidently serving a look-alike, represents a safer and more deployable design.

A second motivation is cost. Queries that reach an LLM may be routed to different providers, but a complexity heuristic alone does not establish answer quality or measured monetary savings. Those claims require provider-answer evaluation and real pricing data.

1.3 Problem Definition

The core problem addressed in this project is that contemporary semantic caches for LLMs are **fixed-threshold and unverified**, which makes them unsafe: they trade correctness for savings in a way the operator cannot tune away. Specifically, the problem decomposes into the following sub-challenges:

1. **Threshold brittleness.** A single global similarity threshold cannot simultaneously maximise reuse and avoid false hits, because the correct strictness is query-dependent. The optimal threshold also shifts with the embedding model, so a value tuned for one embedder is wrong for another.

2. **Unverified hits (the false-hit problem).** Similarity above a threshold is not proof of semantic equivalence. Queries differing only in a number, a negation, or a named entity embed very close together yet require different answers, and a naive cache serves the wrong one.

3. **Uncacheable (volatile) queries.** Time-sensitive queries (*"weather today"*, *"current stock price"*) must never be served from cache, but a similarity-only cache has no notion of staleness.

4. **Latency of the cache itself.** A cache is only worthwhile if the lookup is dramatically faster than the LLM call it replaces; a naive linear scan over stored vectors degrades as the cache grows.

5. **Uniform-cost routing.** Sending every cache-missed query to a single expensive model wastes money on queries a cheap model would answer correctly.

6. **Production safety.** A shared cache in front of an LLM introduces multi-tenant data-leakage, cost-exhaustion, and reliability risks that a research prototype typically ignores.

1.4 Project Objectives

To address the above, this project implements **VeriGate**, an LLM gateway prototype, and begins evaluating its cache decisions. The main objectives are:

- **Adaptive similarity threshold.** To raise or lower the effective threshold *per query* based on the density of the query's neighbourhood in embedding space, so that ambiguous regions are treated more strictly than isolated ones (Section 4.8.1).
- **A query-checking semantic cache.** To interpose structural checks and optional bidirectional question NLI between a candidate hit and the served answer. This checks question compatibility, not the correctness of the cached answer (Section 4.8.2).
- **Volatility detection.** To classify time-sensitive queries and bypass the cache for them, preventing stale answers (Section 4.8.3).
- **A sub-linear cache-hit path.** To replace the naive linear scan with a vectorised in-process index (and an optional distributed RediSearch/HNSW backend [8, 13]) so that the hit path stays an order of magnitude faster than an LLM call as the cache scales (Section 5.5).
- **A cost-aware model cascade (second contribution).** To score query complexity and route easy queries to a cheap model, escalating only hard ones to a strong model, cutting cost while preserving quality on hard queries [3] (Section 4.9).
- **Operational safeguards.** Add per-tenant cache isolation, per-key daily request admission limits, a provider circuit breaker, Prometheus/Grafana observability, and Docker packaging (Chapter 5). Public deployment still needs separate validation.

1.5 Scope of the Project

VeriGate is scoped as a self-hostable HTTP gateway that sits in front of one or more OpenAI-compatible LLM providers (it is verified live against Google Gemini and works unchanged with OpenAI). Its intelligence operates on the *text* of queries and answers; it does not perform image, audio, or multi-modal caching. The default embedding model is the 384-dimensional `all-MiniLM-L6-v2` sentence encoder [15]; the architecture is embedder-agnostic and exposes a fast lexical fallback used for deterministic testing.

The evaluation is conducted on a controlled, hand-curated benchmark of positive (equivalent) pairs, hard negatives grouped by trap type (entity swap, number swap, negation, related topic), volatile queries, and stable controls. This design deliberately isolates the *mechanism* being studied; the *relative* improvement of VeriGate over the fixed-threshold baseline is the result, and the absolute numbers are acknowledged to be dataset-dependent (Chapter 6). Large-scale learned routing and threshold classifiers, and a direct empirical study on a million-scale corpus, are identified as future work (Chapter 7). The gateway is deployed as a containerised web service and exposes an operator dashboard for interactive testing.

1.6 Project Timeline

Development followed an iterative approach from gateway plumbing through cache experiments and operational hardening. Figure 1.1 summarises the historical plan; Chapter 6 now separates recorded smoke measurements from untested research claims.

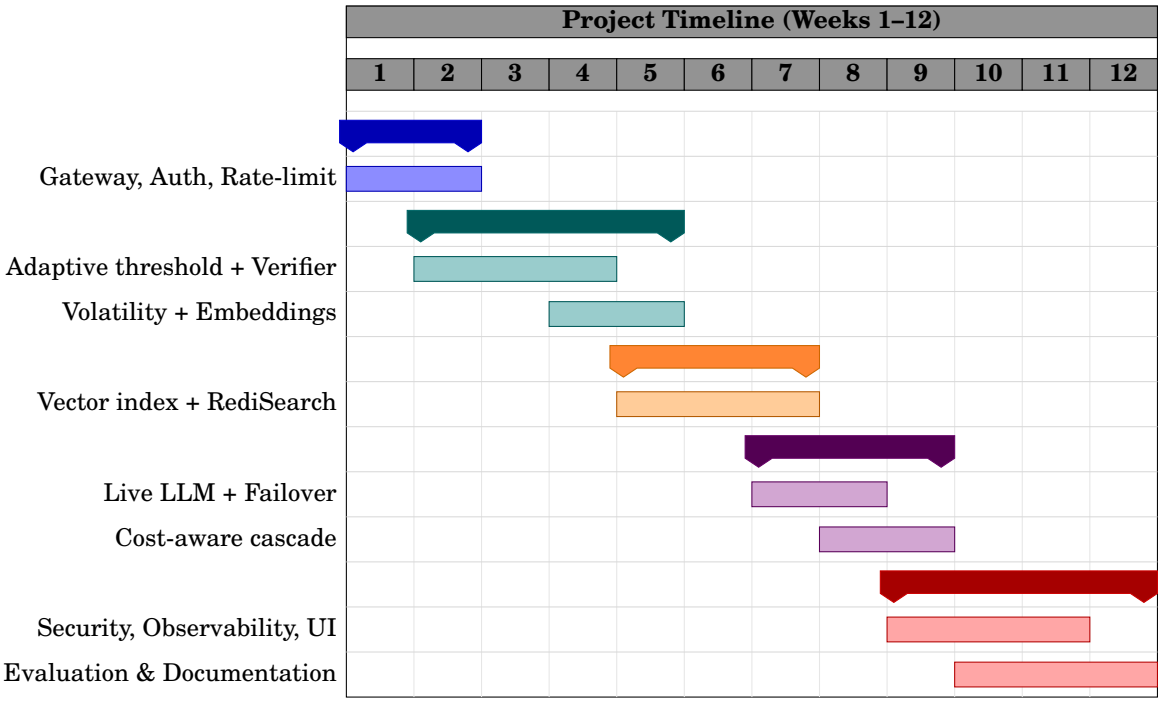


Figure 1.1: VeriGate development timeline across twelve weekly sprints, colour-coded by phase: Foundation (blue), Cache Core (teal), Performance (orange), Providers and Cascade (purple), and Hardening and Evaluation (red).

The gateway and the pair evaluator share a `CachePolicy` decision function, but the pair evaluator does not reproduce retrieval, changing cache contents, or provider answers. It is therefore a limited check rather than an end-to-end validation of the live semantic mode.

Chapter 2

Literature Survey

2.1 Scope of the Comparison

Semantic answer caching stores a response with a prior request and retrieves it for a later, related request. The central question is whether the *particular stored answer* is reusable, not merely whether two question embeddings are close. The systems below already cover important parts of adaptive and verified caching; VeriGate does not claim to have invented either idea.

2.2 Directly Relevant Systems

GPTCache is an open-source cache framework with configurable embeddings, vector storage, similarity evaluation, thresholds, and an optional LLM verifier in post-processing. It is therefore inaccurate to describe GPTCache as a system that can only use a fixed threshold or can never verify a candidate. The feature set is documented in its `https://github.com/zilliztech/GPTCache/blob/main/examples/README.md`.

vCache, published as an ICLR paper, learns cache-entry-specific decision boundaries online and studies user-specified error bounds under its stated model and assumptions. It compares responses obtained during exploration with cached responses and evaluates several benchmark distributions. VeriGate's hand-set query threshold has no analogous guarantee. The paper is available at `https://arxiv.org/html/2502.03771v5`.

Krites studies a static/dynamic cache with a similarity grey zone. It runs an LLM judge asynchronously after a miss, then promotes approved static responses into the dynamic cache. Its objective includes preserving critical-path latency and accounting for judge cost. VeriGate does not implement this policy. The paper is available at `https://arxiv.org/html/2602.13165v2`.

Table 2.1: Verified scope of directly relevant caching approaches. A checkmark describes a reported method, not an independently reproduced result in this project.

System	Match/decision	Verification	Error and cost scope
GPTCache	Configurable embedding and similarity evaluation	Optional LLM post-processor	No guarantee claimed here; configuration dependent
vCache	Online, cache-entry-specific boundary	Exploration compares generated and cached responses	Error bound under published assumptions; exploration has inference cost
Krites	Threshold plus grey-zone trigger	Asynchronous LLM judge after a miss	Studies static-answer coverage, error and judge overhead
VeriGate (current)	Hand-set query threshold; exact mode by default	Structural query checks; optional bidirectional question NLI	Small pair fixture only; no answer-level or formal risk validation

2.3 Research Position

The defensible next question for VeriGate is whether an *answer-aware and cost-aware* selective verifier can improve the trade-off between valid reuse, incorrect reuse, and verification latency on a held-out workload. This is a hypothesis, not a current result. It overlaps with prior verification work, so any eventual contribution must be stated narrowly: a particular policy and evaluation protocol, with direct comparisons and negative results included. Query-only NLI is a question-equivalence signal, not proof that a cached answer is still correct.

Chapter 3

Requirement Engineering

3.1 Introduction

Requirement engineering translates the goals of Chapter 1 into precise, verifiable specifications. For VeriGate the requirements divide into those governing the caching intelligence (the adaptive threshold, verifier, and volatility logic), those governing the gateway's role as a reliable proxy in front of paid LLM providers, and those governing production safety (isolation, budgeting, observability). Functional requirements state *what* the system must do; non-functional requirements state *how well* it must do it.

3.2 System Analysis

3.2.1 Limitations of the Existing System

Before designing VeriGate, existing semantic-caching and gateway solutions were analysed. The principal limitations identified were:

1. **Single fixed threshold.** Systems such as GPTCache [1] expose one global similarity threshold. It cannot be tuned to be simultaneously permissive for paraphrases and strict for look-alikes, and it is silently wrong when the embedding model changes.
2. **No hit verification.** A candidate hit above the threshold is served without any check that the two queries actually mean the same thing, producing false hits on number-, entity-, and negation-differing queries.
3. **No notion of staleness.** Time-sensitive queries are cached and served like any other, returning stale answers for volatile information.
4. **Cross-tenant leakage.** A global semantic cache lets one client's question match and receive another client's answer — a confidentiality breach unique to *semantic* (as opposed to exact) caching, and the single most serious risk for a shared gateway.
5. **Cost-blind routing.** Every cache-missed query is sent to one model, overpaying on queries a cheaper model would answer correctly.
6. **Operational fragility.** A dead or slow provider is retried on every request, and there is no spend cap, so a burst of unique queries can exhaust both latency budgets and money.

3.2.2 Feasibility Study

- **Technical Feasibility:** The system is built entirely on mature open-source components — FastAPI [12], the sentence-transformers encoder [15], Redis, and standard scientific-Python libraries — and requires no GPU for inference. The default in-process vector index runs in a single Python process, and the RediSearch backend uses a standard Redis Stack container. The project is therefore technically highly feasible.
- **Operational Feasibility:** The gateway is a drop-in HTTP proxy: an existing application changes only the base URL of its LLM client. An operator dashboard makes the cache's behaviour observable and testable without writing code, easing adoption.
- **Economic Feasibility:** Core libraries are open-source, but hosting, Redis, model inference, and provider calls have deployment-dependent costs. Cache-hit and routing savings remain hypotheses until measured on representative traffic with current prices.

3.3 System Requirements

3.3.1 Hardware Requirements

- **Client (application using the gateway):** any host able to make HTTPS requests; no local computation.
- **Development environment:**
  - **Processor:** Intel Core i5 / AMD Ryzen 5 or better.
  - **RAM:** 8 GB minimum; the sentence-transformer model and the in-process vector matrix are the main consumers.
  - **Storage:** SSD recommended for fast model load and container builds.
- **Production server (minimum):**
  - **CPU:** 2 virtual cores.
  - **RAM:** ~1 GB for the lexical-embedding lite mode; ~1.5–2 GB when the MiniLM model is loaded.
  - **Storage:** 20 GB block storage for the container image and model cache.

3.3.2 Software Requirements

The stack is uniformly Python-based to keep the caching layer and the web server in one runtime.

- **Operating System:** Agnostic (Windows 11 during development; Linux containers for deployment).
- **Language:** Python 3.11+ (the reference environment is Python 3.13).
- **Web framework:** `FastAPI + uvicorn` (ASGI) for asynchronous request handling and streaming (Server-Sent Events) [12].
- **Embeddings:** `sentence-transformers` with `all-MiniLM-L6-v2` [15]; `torch` (CPU build) as the backend; a hash-based lexical embedder as a dependency-free fallback for tests.

- **Verifier:** a compact NLI cross-encoder (`cross-encoder/nli-deberta-v3-xsmall`) [5] for the optional Tier-2 check.
- **Cache / state:** `redis` (with `fakeredis` as an in-process default for development and tests); Redis Stack for the RediSearch/HNSW vector backend [13].
- **Observability:** `prometheus-client` for metrics; Prometheus + Grafana for scraping and dashboards [11].
- **Packaging:** Docker and Docker Compose; `pydantic-settings` for typed configuration.

3.4 Functional Requirements

1. **Authenticated chat endpoint.** The gateway must accept a chat request bearing a valid API key and return a completion; requests without a valid key must be rejected with HTTP 401.
2. **Streaming.** The gateway must support token streaming (SSE) as well as a non-streaming JSON response.
3. **Semantic cache lookup.** For each query the system must embed the query, find its nearest cached neighbour within the caller's tenant, and, subject to the policy below, serve the stored answer as a HIT rather than calling the provider.
4. **Adaptive threshold.** The effective similarity threshold must be adjusted per query as a function of neighbourhood density, rather than being a single global constant.
5. **Self-verification.** Before serving a candidate hit, the system must run a Tier-1 structural verifier (numbers, negation, named entities) and, when enabled, a Tier-2 NLI verifier; a candidate that fails verification must be rejected as a false hit and treated as a MISS.
6. **Volatility bypass.** Queries classified as time-sensitive must bypass the cache (never served from and never written to it).
7. **Fast lookup.** The cache-hit path must use a vectorised index (in-process or RediSearch) so that lookup remains far faster than a provider call as the cache grows.
8. **Cost-aware cascade.** When enabled, the gateway must score query complexity and route easy queries to a cheap model, escalating hard queries to a strong model, keeping the unused tier as failover.
9. **Provider abstraction and failover.** The gateway must front OpenAI-compatible providers (OpenAI, Gemini) selected by configuration, and must fail over to a mock provider so a provider outage never hard-fails a request.
10. **Per-tenant isolation.** Each API key must map to a tenant whose cache is isolated; one tenant must never receive another tenant's cached answer.
11. **Per-key request budget.** The system must cap admitted cache misses and bypasses per key per day; over-budget requests receive HTTP 429, and cache hits do not count. This does not cap provider retries, tokens, or monetary spend.
12. **Rate limiting.** The system must throttle bursts per API key via a token bucket, returning HTTP 429 when the bucket is empty.
13. **Observability endpoints.** The system must expose `/health` (live configuration) and `/metrics` (Prometheus), and an operator dashboard at `/ui` for interactive testing and a cache-clear admin action.

3.5 Non-Functional Requirements

1. **Correctness (primary).** Measure incorrect accepted reuse on independently labelled, held-out answer-aware examples. A configured risk target is not a guarantee; report observed risk and uncertainty separately.
2. **Cache-hit latency.** A cache hit must complete at least an order of magnitude faster than a hosted-LLM call; the target hit path is ~10 ms against an ~800 ms reference call.
3. **Verification overhead.** Tier-1 verification must impose negligible overhead (<1 ms) on the hit path so that safety is effectively free; the cost of Tier-2 must be quantified and its use made optional.
4. **Scalability.** Lookup latency must remain sub-linear in cache size (near-flat and sub-millisecond to at least several thousand entries for the in-process index).
5. **Security.** API-key authentication, secret handling, and per-tenant isolation require tests and deployment review. The daily request budget is an admission control, not a financial limit.
6. **Reliability.** A failing provider must be skipped quickly by a circuit breaker rather than retried on every request; the cache must fail open (a cache error degrades to a provider call rather than an error).
7. **Reproducibility.** The evaluation harness must share the exact `CachePolicy` used by the live gateway, so that measured results equal shipped behaviour; all figures and tables must be regenerable from the repository.

Chapter 4

System Design

4.1 Introduction to System Design

System design translates the requirements of Chapter 3 into a concrete, buildable blueprint. This chapter presents that blueprint at every level an examiner and an implementer need: the guiding design principles and the reusable design patterns that shape the code (Sections 4.2–4.3); the layered runtime architecture (Section 4.4); the behavioural models — use cases, data-flow diagrams, the cache decision, and a request sequence (Sections 4.5–4.7); the design of the novel cache intelligence and the cost-aware cascade (Sections 4.8–4.9); the static structure — modules, data model, and API surface (Sections 4.10–4.12); and finally the deployment topology and failure handling (Sections 4.13–4.14). Throughout, VeriGate is designed as a modular asynchronous proxy in which a single `CachePolicy` decision object is shared between the live server and the evaluation harness, so that the behaviour that is *measured* is exactly the behaviour that is *shipped*.

4.2 Design Goals and Guiding Principles

Every structural choice in VeriGate serves one of six explicit design goals. Stating them first makes the rest of the chapter legible as a set of decisions rather than a list of components.

1. **Correctness over savings.** A wrong cached answer is worse than a missed saving. The design therefore separates the *threshold* decision (does a candidate exist?) from the *equivalence* decision (may it be served?), so that raw similarity can never, by itself, cause an answer to be served.

2. **Fail open, never fail closed.** The cache is an optimisation, not a dependency. Any error in Redis, the vector index, or a provider degrades to the next safe option (a provider call, a failover provider, a mock) rather than surfacing an error to the caller.

3. **Measured equals shipped.** The evaluation harness imports the identical policy object used at run time, eliminating the common gap between a paper's reported behaviour and a system's actual behaviour.

4. **Pluggability at every boundary.** Embedding, vector index, and provider are all swappable behind a stable interface, so a single node and a horizontally-scaled cluster run the same code with different configuration.

5. **Safe by construction for multi-tenancy.** Every piece of state is namespaced by a tenant identifier derived from the API key, so cross-tenant leakage — the defining risk of a shared semantic cache — is structurally impossible rather than merely discouraged.

6. **Observability as a first-class concern.** The same metrics that drive the Grafana dashboards are the evidence base for the evaluation, so the system is instrumented from the first request.

4.3 Design Patterns Employed

The design realises these goals through a small set of well-established software patterns. Table 4.1 names each pattern, where it is applied, and the goal it serves.

Pattern	**Where applied in VeriGate**	**Purpose / goal served**
Gateway / Proxy	The service itself sits in front of the LLM providers	Single control point for caching, routing, auth, budgeting
Strategy (pluggable backend)	Embedding (`minilm`/`hash`), vector index (`inproc`/`redisearch`), provider (OpenAI/Gemini/mock)	Swap implementations by configuration without touching callers (pluggability)
Policy Object	`CachePolicy.decide()` as one pure decision function	Shared by server and evaluator (measured = shipped); unit-testable
Chain of Responsibility	Threshold → Tier-1 → Tier-2 verifier stages; provider failover chain	Cheap checks first; each stage can accept, reject, or defer
Circuit Breaker [9]	`breaker.py` per provider, state in Redis	Skip a dead upstream fast instead of retrying (fail open)
Token Bucket	Per-key rate limiting (atomic Redis)	Bound burst load fairly per client
Decorator / Middleware	Auth, tenant derivation, rate limit, budget wrap each request	Cross-cutting concerns kept out of the core logic

Table 4.1: Design patterns used in VeriGate and the design goal each one serves.

4.4 System Architecture

The gateway is organised into four cooperating layers. Requests descend through them; the cache policy short-circuits the descent whenever a verified hit is available.

- **Layer 1 — Edge / API (FastAPI).** Terminates HTTP, authenticates the API key, derives the tenant, applies rate limiting and the daily request admission budget, and exposes the chat, health, metrics, admin, and dashboard routes. Streaming responses are served as Server-Sent Events.

- **Layer 2 — Cache Intelligence.** The heart of the system: it embeds the query, retrieves the nearest neighbour within the tenant, and applies the `CachePolicy` (adaptive threshold → verifier → decision). It also classifies volatile queries for bypass. On a verified hit it returns the stored answer; otherwise it forwards to Layer 3.

- **Layer 3 — Provider Routing.** Selects the upstream model. With the cascade enabled it scores complexity and chooses the cheap or strong tier; a circuit breaker skips unhealthy providers, and a mock provider is always retained as a terminal failover.
- **Layer 4 — State & Storage.** Redis (or in-process `fakeredis`) holds cache entries, rate-limit buckets, budget counters, and breaker state, all namespaced per tenant. The vector index (in-process NumPy matrix or RediSearch/HNSW) provides the nearest-neighbour lookup.

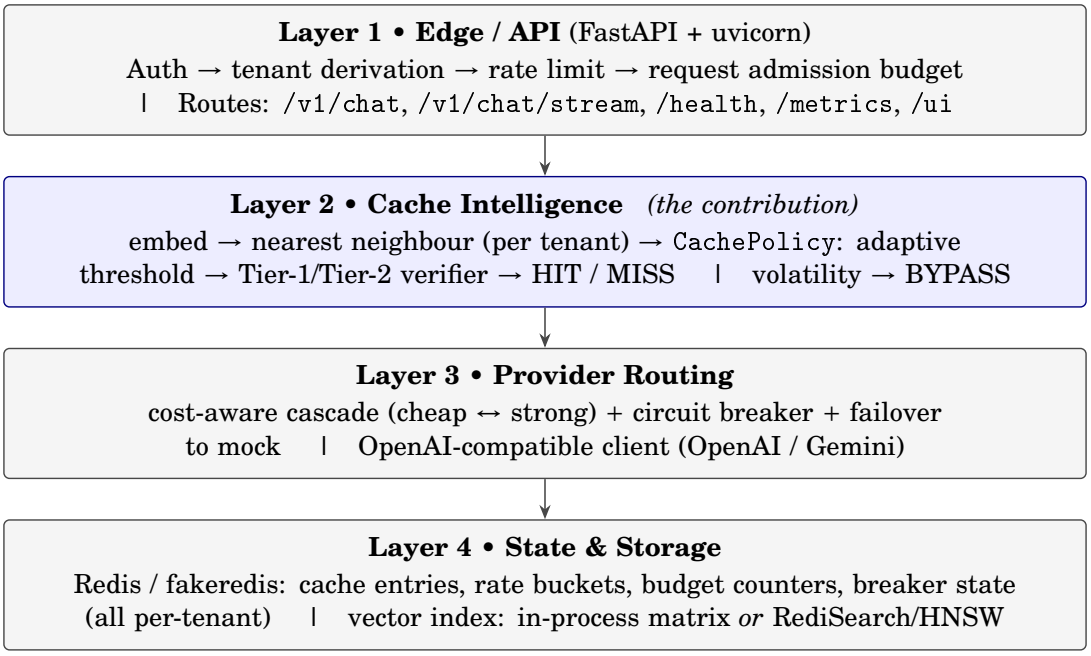


Figure 4.1: VeriGate four-layer architecture. Requests descend from the edge; a verified cache hit in Layer 2 (highlighted) short-circuits the provider call in Layer 3. All state in Layer 4 is namespaced per tenant.

4.5 Use Case Analysis

VeriGate has two human/system actors and one external actor. The **Client Application** (a developer's service) sends chat requests through the gateway. The **Operator** monitors the cache and manages it via the dashboard. The **LLM Provider** (OpenAI / Gemini) is an external actor that fulfils cache-missed calls. Figure 4.2 shows the use cases within the system boundary.

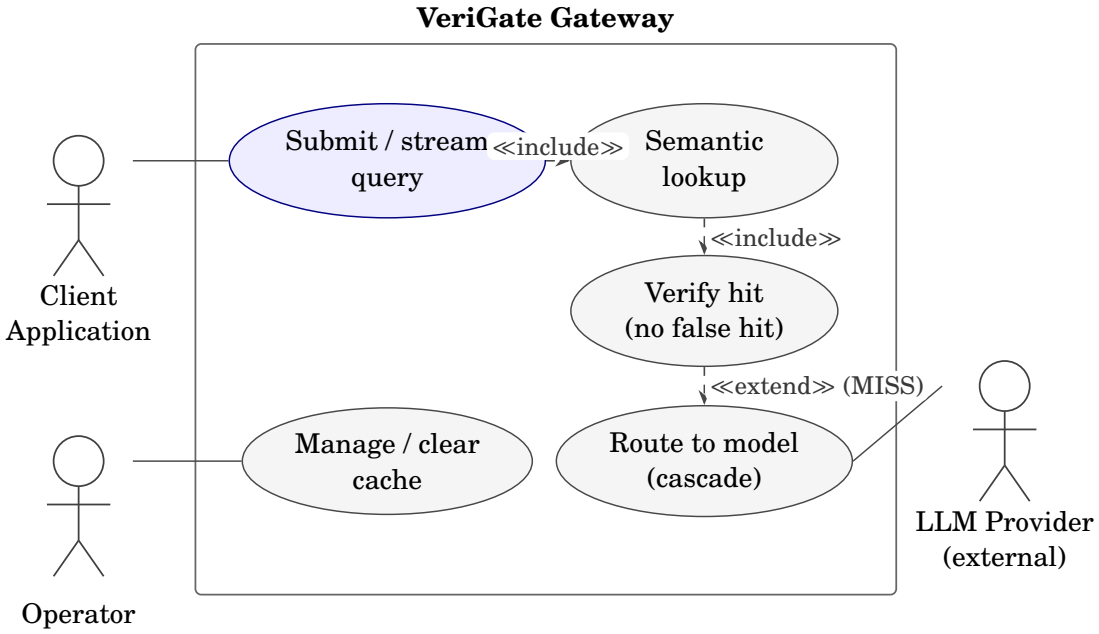


Figure 4.2: UML use-case diagram. The Client Application submits and streams queries; every submission *includes* a semantic lookup and a verify step, and *extends* to a model route only on a miss. The Operator manages and clears the cache; the external LLM Provider fulfils missed calls.

4.6 Data Flow Diagrams

A Data-Flow Diagram (DFD) models how information moves through the system independently of control flow. The Level-0 (context) diagram treats the gateway as a single process; the Level-1 diagram decomposes it into sub-processes and data stores.

4.6.1 Level-0 DFD (Context Diagram)

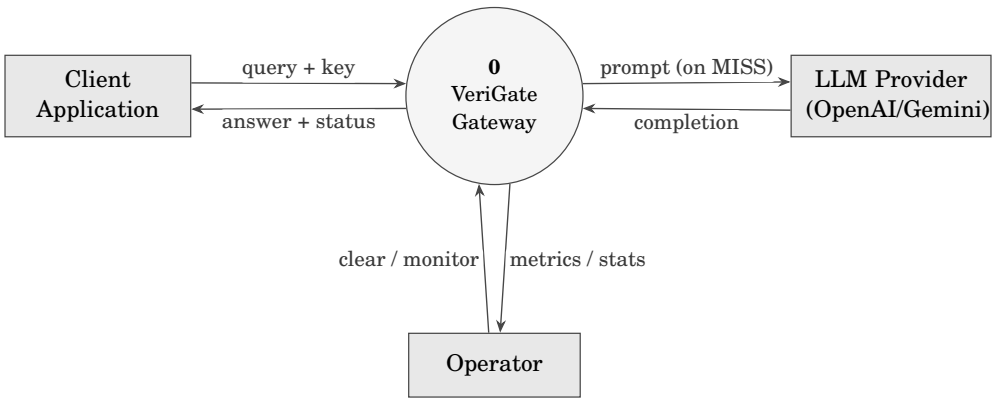


Figure 4.3: Level-0 (context) DFD. The gateway exchanges queries and answers with the Client, prompts and completions with the external LLM Provider (only on a cache miss), and control/metrics with the Operator.

4.6.2 Level-1 DFD (Process Decomposition)

The gateway decomposes into five processes (P1–P5) reading and writing five data stores (D1–D5).

- **P1 Guard:** authenticate key, derive tenant, apply rate limit and request admission budget (reads/writes D3).
- **P2 Embed & Lookup:** embed the query and find the nearest neighbour in the tenant's partition (reads D2, the vector index).
- **P3 Verify & Decide:** read the candidate entry (D1), apply the adaptive threshold and the verifier, classify volatility, and emit HIT/MISS/BYPASS.
- **P4 Route & Call:** on a miss, run the cascade, skip open breakers (reads/writes D4), call the provider, and store the answer (writes D1, D2).
- **P5 Observe:** record counters and latencies (writes D5) exposed at `/metrics`.

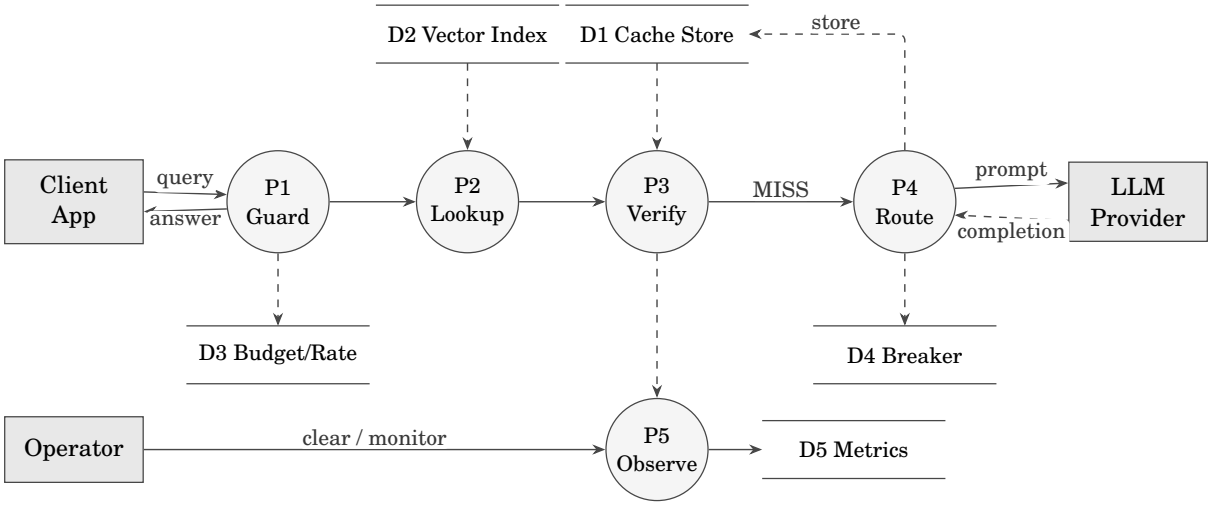


Figure 4.4: Level-1 DFD. The main data flow runs left to right along one lane (Client → P1 → P2 → P3 → P4 → Provider); data stores connect by vertical (orthogonal) flows. A verified hit in P3 returns the answer to the client; only a miss reaches P4 and the external provider, whose answer is written back to the cache (D1).

4.7 Request Lifecycle and the Cache Decision

The defining logic of VeriGate is the decision, taken for every query, among four outcomes: **BYPASS** (volatile, never cached), **HIT** (a verified equivalent answer is served), **blocked** (a candidate above threshold is rejected by the verifier and demoted to a miss), and **MISS** (the provider is called and the answer is stored). Figure 4.5 formalises this flow; Figure 4.6 then traces the two dominant paths — a cold MISS and a warm HIT — as a sequence of messages between components.

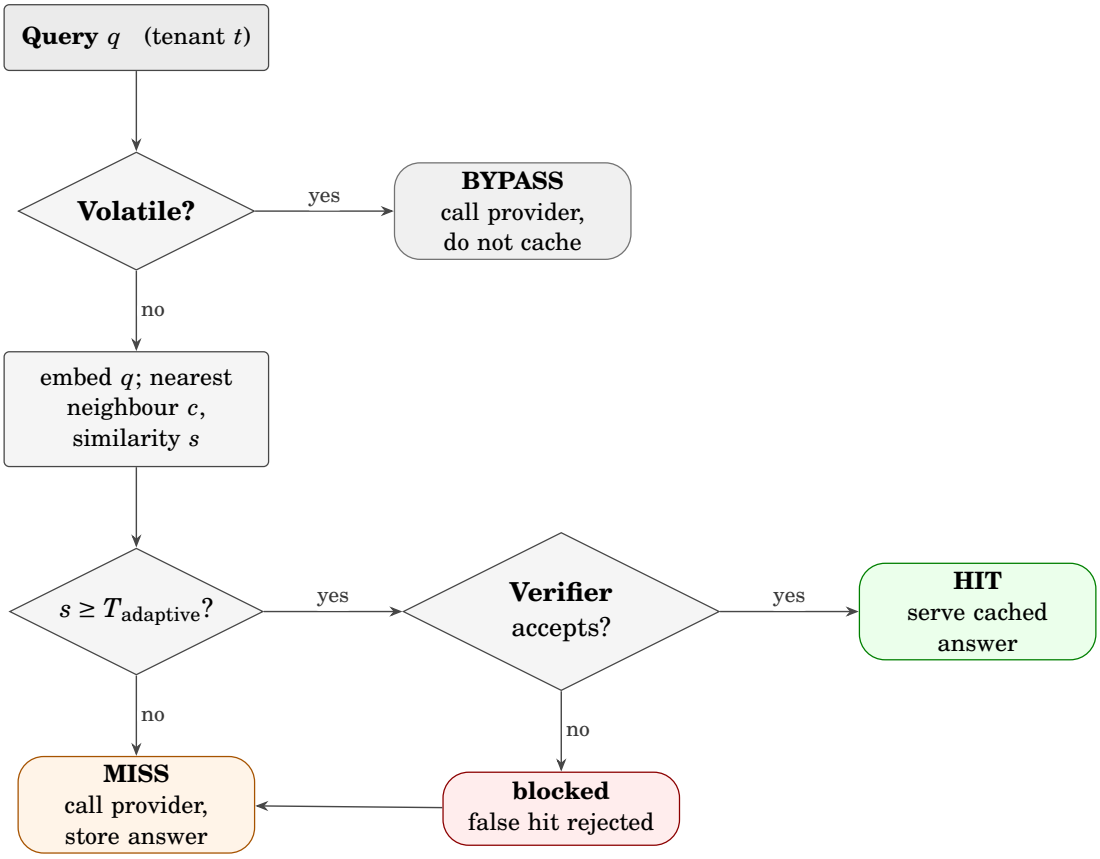


Figure 4.5: The four-outcome cache decision. A candidate whose similarity clears the adaptive threshold is still subjected to the verifier; only a verified candidate is served as a HIT, while a rejected one is demoted to a MISS ("blocked"). Volatile queries bypass the cache entirely.

4.7.1 Request Sequence (HIT vs MISS)

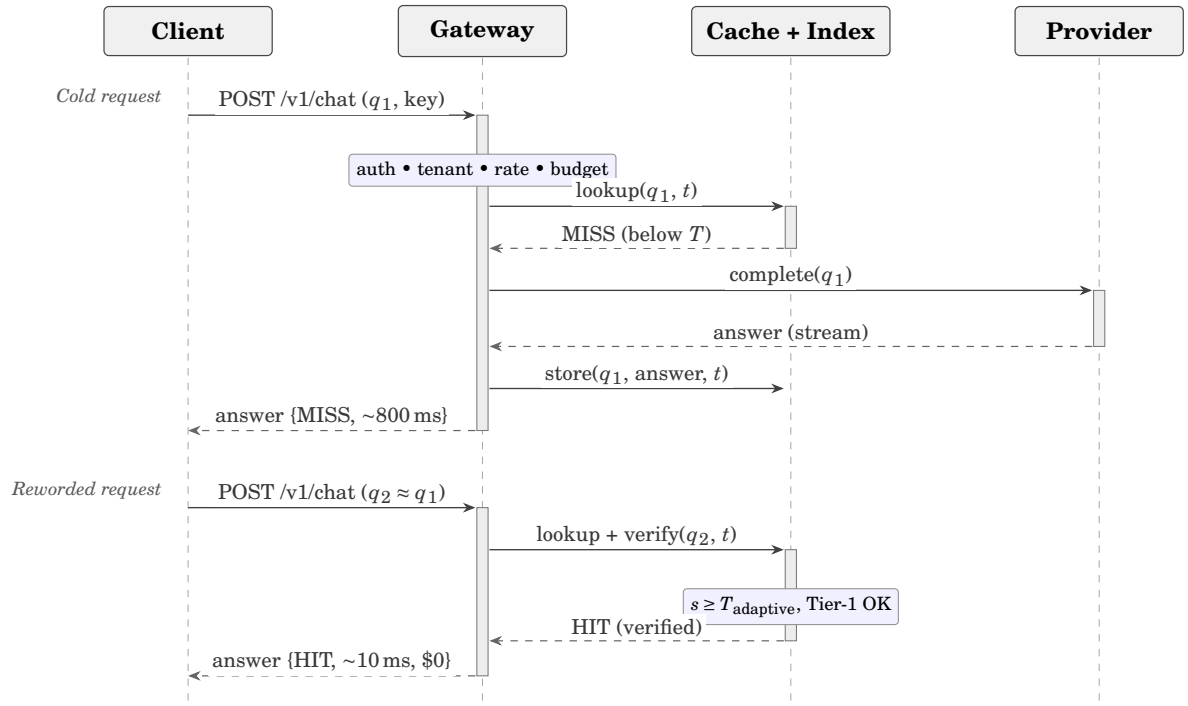


Figure 4.6: Request sequence. A cold query misses the cache, is answered by the provider (~800 ms), and is stored. A subsequently reworded query clears the adaptive threshold, passes verification, and is served from cache (~10 ms, $0) without touching the provider — the core cost/latency win. Solid arrows are calls; dashed arrows are returns; bars are activations.

4.8 Cache Intelligence Design

The cache intelligence is composed of three sub-designs that together constitute the experimental cache policy.

4.8.1 Adaptive Similarity Threshold

Rather than a global constant T_{base}, the effective threshold is raised in *dense* neighbourhoods, where many stored queries crowd around the incoming one and the risk of a look-alike collision is highest. Let s be the top-1 cosine similarity and let a local density signal $\rho \in [0,1]$ measure how close the runner-up neighbours are to the top candidate. The adaptive threshold is

$$T_{\text{adaptive}} = \min\big(T_{\max},\, T_{\text{base}} + \alpha \cdot \rho\big), \tag{4.1}$$

where T_{base} is the calibrated base threshold for the embedding model (0.72 for `all-MiniLM-L6-v2`, see Chapter 6), α is a sensitivity coefficient, and $T_{\max}$ caps the strictness. In sparse regions the threshold stays near T_{base} so genuine paraphrases still hit; in dense regions it rises so ambiguous look-alikes are held to a higher bar. This directly attacks the related-topic false hits that a fixed threshold cannot separate without also rejecting paraphrases.

4.8.2 Two-Tier Self-Verifier

The verifier is the mechanism that distinguishes VeriGate from all fixed-threshold caches. It runs *after* a candidate clears the threshold and decides whether the two queries truly mean the same

thing.

Tier-1 (structural, near-free). Most dangerous false hits differ in a small set of high-signal features. Tier-1 extracts and compares:

- **Numbers** — a query mentioning "5 gb" must not be served the "50 gb" answer;
- **Negation** — "how to enable X" must not be served "how to disable X";
- **Named entities** — "capital of Austria" must not be served "capital of Australia".

If the candidate and the query disagree on any of these, the hit is rejected with a reason code (e.g. `number_mismatch`). Tier-1 deliberately does *not* use blanket word-overlap, which an early design showed wrongly rejects legitimate paraphrases that share few words; it checks only the load-bearing tokens.

Tier-2 (NLI, optional). For correctness-critical domains, an NLI cross-encoder [5, 2] tests *bidirectional* entailment: the candidate is accepted only if $q \Rightarrow c$ and $c \Rightarrow q$. It is gated behind a configuration flag because it is comparatively expensive and, as Chapter 6 shows, largely redundant once the adaptive threshold is active.

4.8.3 Volatility Detection

A volatility classifier flags queries whose correct answer changes with time (weather, prices, "today", "current", "now"). Such queries are marked BYPASS: they are never served from and never written to the cache, eliminating the staleness class of errors entirely.

4.9 Cost-Aware Cascade Design

For queries that miss the cache, a complexity score $\kappa(q) \in [0,1]$ (based on length, structure, and difficulty cues) decides the model tier: if $\kappa(q) < \tau$ the query is routed to the cheap model, otherwise it is escalated to the strong model. The unused tier is retained as failover. This realises the FrugalGPT cascade idea [3] in a transparent, dependency-free form.

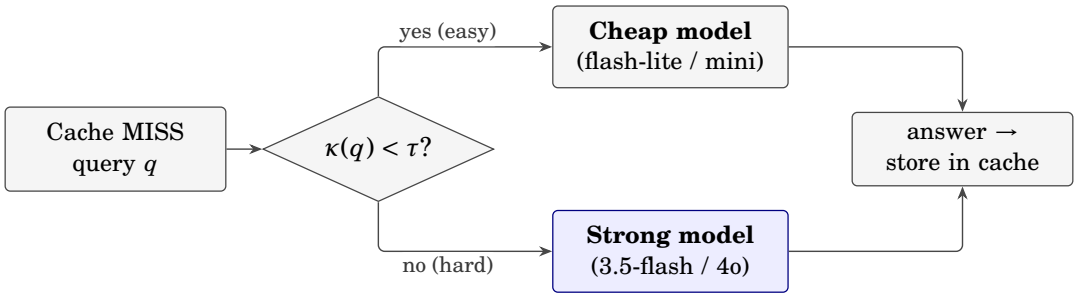


Figure 4.7: Cost-aware cascade. A transparent complexity score routes easy queries to the cheap model and escalates only hard ones to the strong model; the unused tier remains as a failover.

4.10 Component and Module Design

The static structure follows the layers of Figure 4.1. Table 4.2 maps each source module to its responsibility and its principal collaborators, showing that the cache decision logic is

concentrated in app/cache/ behind a single policy object.

Module	Responsibility	Layer
app/main.py	Routes, auth, tenant derivation, rate limit, budget wiring	1 (Edge)
app/config.py	Single typed Settings (all env knobs)	cross-cutting
app/cache/policy.py	CachePolicy.decide() — the shared decision object	2 (Cache)
app/cache/threshold.py	Adaptive threshold (Eq. 4.1)	2
app/cache/verifier.py	Tier-1 structural + Tier-2 NLI verification	2
app/cache/volatility.py	Volatility (staleness) classifier	2
app/cache/semantic_cache.py	Per-tenant embed / lookup / store; in-process index	2, 4
app/cache/redisearch_store.py	RediSearch/HNSW backend with tenant TAG	4
app/complexity.py	Query complexity score for the cascade	3
app/providers/registry.py	Build providers / cascade; route_stream	3
app/providers/openai_compat.py	OpenAI/Gemini streaming client	3
app/providers/breaker.py	Per-provider circuit breaker (Redis state)	3, 4
app/budget.py	Per-key daily request admission budget	1, 4
app/metrics.py	Prometheus counters and histograms	cross-cutting
app/static/dashboard.html	Operator dashboard served at /ui	1

Table 4.2: Module responsibilities mapped to architectural layers. The contribution is concentrated in app/cache/, reached through the single CachePolicy object.

4.11 Data Model and Tenant Namespacing

All state is partitioned per tenant to close the cross-tenant leakage risk. The tenant identifier is derived deterministically from the API key so that keys never appear in storage:

$$\texttt{tenant_id} = \texttt{"t_"} + \mathrm{SHA256}(\texttt{api_key})[:16]. \tag{4.2}$$

Table 4.3 lists the namespaced keys. The in-process index is a dictionary of per-tenant NumPy matrices; the RediSearch backend stores a tenant TAG field and pre-filters every k-nearest-neighbour query by it.

Purpose	Key pattern	Notes
Cache entry ids	`cache:ids:<tenant>`	set of entry ids for the tenant
Cache entry	`cache:entry:<tenant>:<id>`	stored query, answer, vector; optional TTL
Rate-limit bucket	`ratelimit:<api_key>`	token-bucket state (atomic)
Spend budget	`budget:<api_key>:<day>`	provider-call counter; hits are free
Circuit breaker	`breaker:open:<provider>`	open/closed state + failure count
RediSearch doc	`vec:<tenant>:<id>`	HNSW-indexed vector with `tenant` TAG

Table 4.3: Namespaced state in Redis. Every cache and index key is scoped by `tenant_id`, so one tenant can never read another's entries.

4.12 API Route Design

Route	Method	Auth	Purpose
`/v1/chat`	POST	Yes	Chat completion (non-streaming JSON) with cache lookup.
`/v1/chat/stream`	POST	Yes	Streaming completion (Server-Sent Events).
`/health`	GET	No	Liveness + live configuration (backends, provider).
`/metrics`	GET	No†	Prometheus metrics (requests, cache outcomes, latency).
`/admin/cache/clear`	POST	Yes	Clears *only* the caller's tenant cache.
`/` , `/ui`	GET	No	Operator dashboard (interactive testing).

Table 4.4: Primary API routes. † `/metrics` is public in the demo build and is flagged for lock-down before any multi-tenant public deployment (Chapter 5, security).

4.13 Deployment Topology

The same image runs from a laptop to a cluster by configuration alone. Figure 4.8 shows the production topology: clients reach one or more stateless gateway replicas behind a load balancer; the replicas share one Redis (or Redis Stack) for cache, budgets, and breaker state, and call the external provider on misses; Prometheus scrapes each replica and Grafana renders the dashboards. Because all state lives in shared Redis, replicas are horizontally scalable and interchangeable.

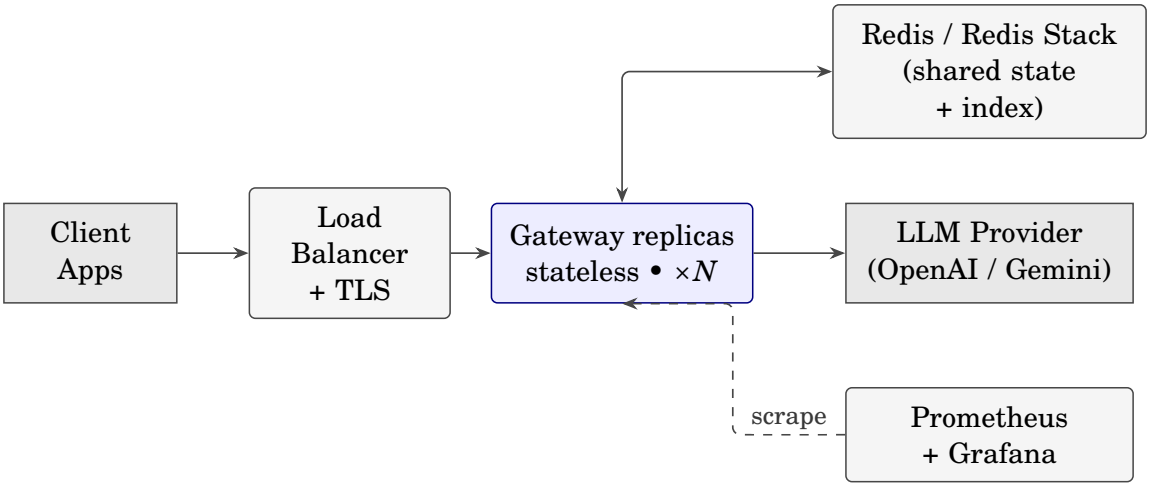


Figure 4.8: Production deployment topology. Stateless gateway replicas ($\times N$) sit behind a TLS-terminating load balancer and share one Redis for all state, so the cache, budgets, and breaker are consistent across replicas; Prometheus/Grafana observe every replica. Each connector uses an orthogonal path on its own side of the gateway.

4.14 Failure-Mode and Error-Handling Design

The gateway is designed to degrade gracefully rather than fail:

1. **Cache fails open.** If Redis or the vector index errors, the request degrades to a normal provider call rather than returning an error — the cache is an optimisation, never a dependency.
2. **Provider failover.** If the selected provider fails (outage, bad key), the router tries the next candidate and finally a mock provider, so a request never hard-fails on a provider problem.
3. **Circuit breaker.** After N failures within a window a provider's breaker opens and the router skips it for a cooldown, preventing repeated slow failures against a dead upstream [9].
4. **Budget and rate limits.** Over-budget or over-rate requests are rejected cleanly with HTTP 429; cache hits are exempt from the budget so the cache extends, rather than consumes, the allowance.
5. **TTL and cleanup.** Optional per-entry TTL bounds memory and self-heals staleness; dangling ids are removed on index rebuild.

Table 4.5 summarises the fault-tolerance design as a fault/detection/response matrix.

Fault	**Detection**	**System response**
Redis / index unavailable	exception on lookup/store	degrade to direct provider call (fail open)
Provider outage / bad key	non-2xx or timeout	failover to next provider, then mock
Repeated provider failures	breaker failure counter	open breaker, skip provider for cooldown
Cost-exhaustion (bill shock)	per-key daily counter	HTTP 429; cache hits stay free
Request burst	token bucket empty	HTTP 429 (rate limited)
Stale cached answer	TTL expiry / volatility flag	entry expires; volatile queries bypass

Table 4.5: Fault-tolerance matrix: how each anticipated fault is detected and handled without surfacing an error to the caller.

Chapter 5

Implementation

5.1 Introduction

VeriGate is implemented in Python as an asynchronous FastAPI application. This chapter describes the concrete modules, the key engineering decisions and their justifications, and the algorithms that realise the design of Chapter 4. The implementation is organised into the following modules: `app/main.py` (routes, auth, tenant derivation, rate limit, budget), `app/config.py` (typed settings), `app/cache/` (the contribution: `semantic_cache.py`, `policy.py`, `threshold.py`, `verifier.py`, `volatility.py`, `redisearch_store.py`), `app/complexity.py` (cascade routing score), `app/providers/` (provider abstraction, cascade registry, circuit breaker), `app/budget.py` (request admission budget), `app/metrics.py` (Prometheus counters), and `app/static/dashboard.html` (operator UI).

5.2 Architectural Decisions

Why FastAPI + uvicorn rather than Flask? The gateway is I/O-bound: most of its time is spent awaiting upstream LLM calls. FastAPI's ASGI foundation [12] allows a single worker to handle many concurrent in-flight requests without threads, and it natively supports streaming responses (Server-Sent Events), which are essential for forwarding token streams from the provider. Pydantic-based request validation rejects malformed bodies with HTTP 422 for free.

Why sentence-transformer embeddings rather than TF-IDF? A semantic cache must match by *meaning*. Sparse TF-IDF vectors [16] encode lexical overlap, so two paraphrases with few shared words are nearly orthogonal and would never hit. Dense sentence embeddings [14] place paraphrases close together regardless of surface form. `all-MiniLM-L6-v2` [15, 18] was chosen for its balance of quality and low CPU latency; a deterministic hash-based lexical embedder is retained purely as a dependency-free fallback for the test suite.

Why an in-process vector index as the default, with RediSearch optional? For a single node the fastest possible lookup is a single matrix–vector product held entirely in RAM, with no network hop; this is the default `inproc` backend. For horizontally-scaled deployments that must share one cache across replicas, or for caches too large for one process's memory, a RediSearch/HNSW backend [8, 13] is provided behind a configuration switch. This gives the right default for the common case without foreclosing scale.

Why Redis (with fakeredis) for state? Redis provides atomic operations for the rate-limit token bucket and request counters, TTLs for cache entries, and a shared store for breaker state across replicas. For development and tests, `fakeredis` offers the same interface in-process with no server to run, so the whole system is exercisable with a single `pytest` invocation.

Why keep a mock provider permanently in the chain? Placing a free mock provider as the terminal failover means a provider outage or a bad key degrades to a valid (if synthetic) response rather than a 500 error, satisfying the graceful-degradation requirement and making the system demonstrable with no API key at all.

5.3 Configuration

All behaviour is controlled by a single typed `Settings` object (`pydantic-settings`) read from environment variables, so the same image runs in every environment by changing configuration only. Key knobs include `REDIS_URL`, `CACHE_SIMILARITY_BASE` (0.72), `EMBEDDING_BACKEND` (`minilm/hash`), `VECTOR_BACKEND` (`inproc/redisearch`), `VERIFIER_NLI` (default off), `LLM_PROVIDER`, `CASCADE_ENABLED`, `DAILY_REQUEST_BUDGET`, the `BREAKER_*` parameters, `CACHE_TTL_SEC`, and `API_KEYS`. Critically, the live gateway and the evaluation harness both import the same `CachePolicy`, so measured behaviour equals shipped behaviour.

5.4 The Cache Policy

The policy is the single decision point of Figure 4.5. Algorithm 1 gives its logic. It is intentionally small and pure so that it can be unit-tested and reused verbatim by the evaluator.

Require: query q, candidate c, similarity s, neighbourhood density ρ, tenant t
Ensure: outcome $\in$ {HIT, MISS, BYPASS} with a reason

if IsVolatile(q) **then**
  return (BYPASS, `volatile`)
end if
$T \leftarrow \min(T_{\max},\ T_{\text{base}} + \alpha\rho)$ ▷ adaptive threshold, Eq. 4.1
if $c = \varnothing$ **or** $s < T$ **then**
  return (MISS, `below_threshold`)
end if
if not Tier1Verify(q, c) **then**
  return (MISS, `verify_rejected:<reason>`) ▷ false hit blocked
end if
if `VERIFIER_NLI` **and not** Tier2NLI(q, c) **then**
  return (MISS, `nli_rejected`)
end if
return (HIT, `verified`)

Algorithm 1: Cache decision (`CachePolicy.decide`)

5.4.1 Adaptive Threshold and Calibration

The base threshold was calibrated empirically. The MiniLM cosine-similarity distribution sits lower than the hash embedder's, so an initial value of 0.82 rejected valid paraphrases; it was recalibrated to **0.72** (Chapter 6). This is itself evidence for the adaptive design: because the

correct threshold is embedding-dependent, hard-coding one is fragile. The adaptive term of Equation 4.1 then lifts the threshold in dense neighbourhoods at run time.

5.4.2 Tier-1 Structural Verifier

Algorithm 2 compares only the load-bearing tokens. An earlier version used a blanket Jaccard word-overlap check, which wrongly rejected paraphrases sharing few words — defeating the purpose of semantic matching. The redesign compares numbers, negation state, and named entities only, deferring subtler distinctions to the optional Tier-2 NLI stage.

Require: query q, candidate c
Ensure: accept (true) or reject with reason
if NUMBERS(q) $\neq$ NUMBERS(c) **then**
    return reject(`number_mismatch`)
end if
if NEGATED(q) $\neq$ NEGATED(c) **then**
    return reject(`negation_mismatch`)
end if
if ENTITIES(q) $\neq$ ENTITIES(c) **then**
    return reject(`entity_mismatch`)
end if
return accept

Algorithm 2: Tier-1 structural verification

5.4.3 Tier-2 NLI Verifier

When `VERIFIER_NLI` is enabled, a compact cross-encoder (`cross-encoder/nli-deberta-v3-xsmall`) [5] scores entailment in both directions; the candidate is accepted only if $q \Rightarrow c$ and $c \Rightarrow q$. Results are memoised per query pair so that a threshold sweep pays the NLI cost at most once per pair. The performance/recall trade-off that makes this stage opt-in is quantified in Chapter 6.

5.5 Semantic Cache and Vector Index

`semantic_cache.py` holds the per-tenant index. `lookup(query, tenant)` embeds the query and computes the nearest neighbour; `store(query, answer, tenant)` appends the new vector. The `inproc` backend keeps, per tenant, one NumPy matrix of stacked unit-normalised vectors, so a lookup is a single matrix–vector product followed by an `argmax` — reducing what was an $O(n)$ parse-and-cosine loop to a sub-millisecond operation (Chapter 6, Phase 9). The `redisearch` backend (`redisearch_store.py`) creates an HNSW index with a `tenant` TAG and issues KNN queries of the form `(@tenant:{t})=>[KNN k @vec $BLOB]`, so the isolation boundary is enforced inside the index itself.

5.6 Provider Abstraction, Cascade, and Circuit Breaker

A single OpenAI-compatible streaming client (`openai_compat.py`) serves both OpenAI and Gemini (via Google's OpenAI-compatibility endpoint), selected by `LLM_PROVIDER` and a key read from the environment (never logged). The registry (`registry.py`) builds either a single provider or, when `CASCADE_ENABLED` is set, a cheap/strong pair. `route_stream` implements Algorithm 3: it

scores complexity, orders the tiers, skips any provider whose breaker is open, and always retains the mock as a terminal failover.

Require: query q, providers (cheap, strong, mock), threshold τ
$\kappa \leftarrow$ COMPLEXITY(q)
order $\leftarrow (\kappa < \tau)$? [cheap, strong, mock] : [strong, cheap, mock]
for each provider P **in** order **do**
  if BREAKEROPEN(P) **then**
    continue
  end if
  try P; **on success** record success, **return** stream
  on failure record failure (may open breaker), continue
end for

Algorithm 3: Cascade routing with breaker and failover

The circuit breaker (`breaker.py`) keeps open/closed state and a failure count in Redis (`breaker:open:<provider>`), shared across replicas, opening after `BREAKER_FAIL_THRESHOLD` failures within `BREAKER_WINDOW_SEC` and closing after `BREAKER_COOLDOWN_SEC` [9].

5.7 Production Hardening

Three features close pre-deployment blockers identified in the threat model:

- **Per-tenant isolation** — the cache and index are namespaced by `tenant_id`; `/admin/cache/clear` clears only the caller's tenant. This closes the top vulnerability (cross-tenant leakage).
- **Per-key daily request admission budget** (`budget.py`) — `check_and_count` increments `budget:<key>:<day>` and returns over-limit as HTTP 429; *cache hits are never counted*, so the cache extends the budget. This closes the economic-DoS (bill-shock) risk.
- **Cache TTL** (`CACHE_TTL_SEC`) — optional per-entry expiry bounds memory and self-heals staleness; dangling ids are cleaned on index rebuild.

5.8 Observability and Operator Dashboard

`metrics.py` exposes Prometheus counters and histograms: `verigate_requests_total`, `verigate_cache_total` (by outcome), `verigate_false_hit_rejected_total`, `verigate_rate_limited_total`, `verigate_budget_exceeded_total`, and `verigate_latency_seconds`. A provisioned Prometheus + Grafana stack (`docker-compose.observability.yml`) scrapes `/metrics` and renders a pre-built dashboard. The single-page operator dashboard (`dashboard.html`), served same-origin at `/ui`, lets a user ask questions and watch the cache badge (HIT/MISS/BYPASS/blocked), the answer, provider, latency, similarity, threshold, and reason, alongside live session statistics — the primary interactive demonstration of the contribution.

5.9 Testing

The system ships with an automated `pytest` suite covering the gateway, verifier, cascade, and hardening. `conftest.py` pins a deterministic configuration (`EMBEDDING_BACKEND=hash`, `VERIFIER_NLI=false`, `LLM_PROVIDER=mock`, `VECTOR_BACKEND=inproc`) so tests are fast and reproducible.

- **Unit:** the verifier rejects number/entity/negation mismatches and accepts genuine paraphrases; the complexity scorer routes easy versus hard queries correctly.
- **Integration:** auth returns 401 without a key; a paraphrase produces a HIT after a MISS; a look-alike above threshold is blocked; a volatile query bypasses; a burst is rate-limited (429).
- **Security / hardening:** `test_tenant_isolation` confirms tenant B cannot read tenant A's cache; the budget blocks over-limit while keeping hits free; the breaker opens after repeated failures and failover still succeeds.

The current test count and results are recorded by the CI run for the evaluated revision; Chapter 6 separates those functional checks from research evidence.

Chapter 6

Results and Evaluation

This chapter distinguishes a small candidate-pair smoke fixture from a final evaluation of answer reuse. The figures and tables below are generated from repository scripts and recorded CSV files. They do not establish a production error rate, a formal risk guarantee, or an original algorithmic result. The default gateway mode is exact matching; semantic matching is an opt-in experimental mode.

6.1 Functional Checks

The default local pytest summary is generated from pytest's JUnit XML. Skipped integration tests need a disposable Redis Stack; the repository's CI exercises that path separately. These counts are functional-check evidence, not a statistical quality result.

Table 6.1: Default local pytest run; skipped integration tests require a disposable Redis Stack.

Passed	Skipped	Failures	Errors
60	4	0	0

6.2 Candidate-Pair Smoke Fixture

The input to this experiment is a stored query and an incoming query, labelled by whether they were intended to be paraphrases. It does not include the actual cached answer or a human judgement that the answer remains usable. The experiment calls the same `CachePolicy.decide` function used by the gateway, but passes zero neighbourhood density and always supplies the designated stored query as the candidate. It therefore tests a decision rule in isolation, not candidate retrieval or a changing cache.

Positive recall divides accepted positive pairs by all positive pairs. Negative acceptance divides accepted negative pairs by all negative pairs. Precision divides accepted positive pairs by all accepted pairs and is undefined when there are no accepts. Neither positive recall nor negative acceptance is an end-to-end workload cache-hit or monetary-savings rate.

Table 6.2: Candidate-pair smoke fixture. Positive recall and negative acceptance use different denominators; this is not workload hit rate or monetary savings.

System	Positive recall	Negative acceptance	Precision	Accepted
Baseline (static T=0.72)	40.0%	46.7%	46.2%	13
+ Adaptive T	33.3%	33.3%	50.0%	10
+ Verifier (Tier-1)	40.0%	13.3%	75.0%	8
VeriGate (Tier-1)	33.3%	0.0%	100.0%	5

The current fixture is deliberately small and was used during development. Thresholds and heuristic weights were not selected on a separate validation split. The displayed differences are descriptive results on this fixture; they cannot establish generalisation or a fair comparison with published systems. A future evaluation must label the reusability of each *specific cached answer*, split related examples by group, tune policies on validation data, and replay a filling cache on a held-out workload.

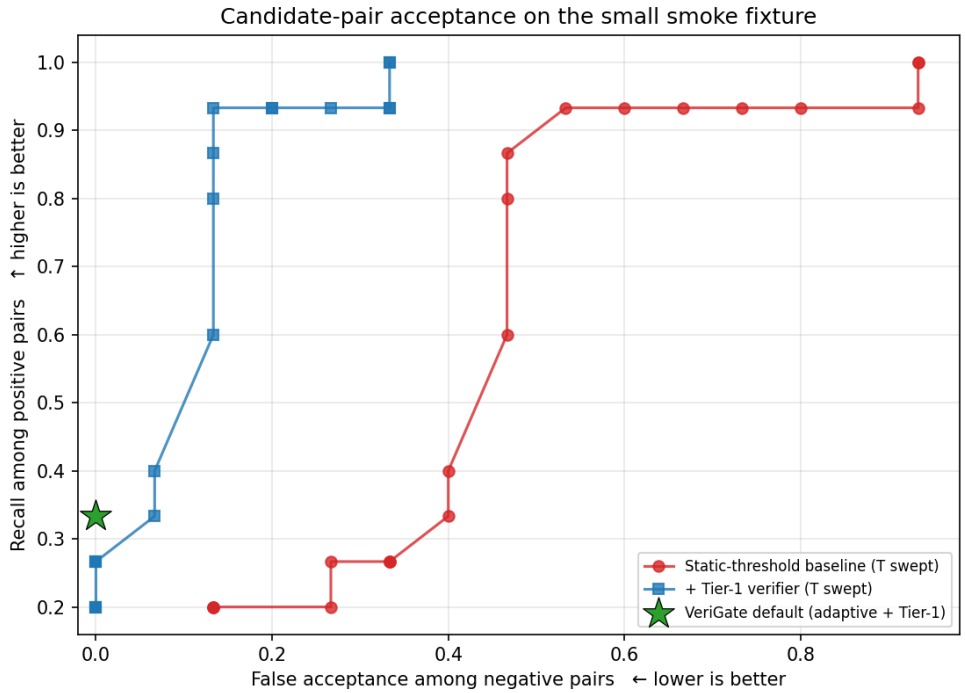


Figure 6.1: Threshold sweep on the candidate-pair smoke fixture. Axes are negative-pair acceptance and positive-pair recall, not real workload risk and savings.

6.3 Stage and Backend Timings

The following archived timing CSV files were produced by the repository's latency and backend scripts. They are stage measurements on a particular machine, not an end-to-end service-level objective. The backend experiment did not measure the crossover size or retrieval recall; no scaling claim follows from it.

Table 6.3: Recorded stage timings in milliseconds; these are not end-to-end provider latency.

Stage	p50	p95	p99
embed	46.51	136.69	348.71
knn_n10	0.05	0.06	1.12
knn_n100	0.05	0.06	0.13
knn_n500	0.08	0.14	0.19
knn_n1000	0.12	0.22	0.33
knn_n2000	0.34	0.71	0.90
tier1_verify	0.03	0.04	0.07
tier2_nli	364.99	882.55	1761.56
hit_tier1_n1000	29.96	47.59	61.88

Table 6.4: Recorded lookup p50 latency in milliseconds; the crossover size was not measured.

Entries	In-process	RediSearch
1000	0.03	3.82
5000	0.20	4.16
10000	0.22	5.58

6.4 Illustrative Cascade Experiment

The cascade experiment uses a hand-labelled easy/hard query list and an assumed relative price for weak and strong providers. Its quality metric is the fraction of queries labelled hard that were routed to the strong provider. It does not compare actual provider answers or measure billed spend. The numbers are useful for testing the router's mechanics only.

Table 6.5: Illustrative cascade cost model. Quality retention is hard-label recall, not judged answer quality.

Threshold	Escalation	Simulated cost reduction	Hard-label recall
0.0	100.0%	0.0%	100.0%
0.35	50.0%	46.7%	100.0%
0.5	42.5%	53.7%	85.0%
0.7	17.5%	77.0%	35.0%
1.0	0.0%	93.3%	0.0%

6.5 Validity and Next Experiment

The smoke fixture is too small for an empirical risk claim. It lacks answer-aware labels, independent annotation, group-separated train/validation/test splits, a realistic request stream, uncertainty intervals, and a real accounting of verification and provider costs. Optional NLI was not part of the default reproduced table; it requires a separately available model and should be reported only when that model ran successfully. The next phase will build a reviewable dataset and evaluation protocol before selecting a learned or risk-controlled policy.

Chapter 7

Conclusion

VeriGate demonstrates a working LLM gateway with exact-match caching as its conservative default and semantic reuse as an opt-in experiment. The gateway implements tenant-aware storage, rate limiting, provider routing, streaming, and observability. Unit and integration tests support specific behaviours, but passing tests do not establish that cached answers are generally safe to reuse.

The current candidate-pair experiment in Chapter 6 is a smoke fixture. It compares query decision policies on a small set of examples chosen during development. It does not evaluate answer compatibility, real retrieval, a changing cache, provider correctness, or measured monetary savings. Its thresholds were not tuned on a separate validation set. Therefore this report does not claim a formal error guarantee, a production quality bound, or that adaptive thresholds and verification were invented by this project.

The next research step is an answer-aware dataset with provenance and independent review, group-separated train, validation, calibration, and held-out test partitions, plus a stream replay that measures retrieval, acceptance, verification cost, and end-to-end cache behaviour. Only after those measurements should a learned or risk-controlled policy be selected. Negative or mixed findings will be reported as results rather than hidden.

Operationally, public deployment still requires production secrets, a real provider, shared Redis, monitoring, load tests, and a release check for the exact revision being deployed. The current work is a tested major-project prototype and a basis for rigorous experimentation, not a completed production service.

Bibliography

[1] Fu Bang. GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS), 2023.

[2] Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. A large annotated corpus for learning natural language inference. In *Proceedings of EMNLP*, pages 632–642, 2015.

[3] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance, 2023.

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[5] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *International Conference on Learning Representations (ICLR)*, 2021.

[6] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[7] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[8] Yu A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[9] Michael T. Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2nd edition, 2018.

[10] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data, 2024.

[11] Prometheus Authors. Prometheus: Monitoring system and time series database. `https://prometheus.io`, 2024. Accessed: 2026-10.

[12] Sebastián Ramírez. FastAPI: A modern, fast web framework for building APIs with python. `https://fastapi.tiangolo.com`, 2024. Accessed: 2026-10.

[13] Redis Ltd. Redis stack and RediSearch: Vector similarity search. `https://redis.io/docs/interact/search-and-query/advanced-concepts/vectors/`, 2024. Accessed: 2026-10.

[14] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992, 2019.

[15] Nils Reimers and Iryna Gurevych. Sentence-transformers: all-MiniLM-L6-v2 model card. `https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2`, 2021. Accessed: 2026-10.

[16] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.

[18] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[19] Kaijie Zhu, Jindong Wang, et al. A survey on efficient inference for large language models, 2024.